

train.py - Block by Block Guide

A simple walkthrough of the main training script for AdvReal adversarial patch generation.

File Structure Overview

```
train.py
├── Imports & Setup (Lines 1-41)
├── init() function (Lines 43-61)
├── PatchTrainer class (Lines 72-402)
│   ├── __init__() - Setup
│   └── train() - Main loop
└── Main entry point (Lines 404-453)
```

1 Imports & Setup (Lines 1-41)

```
# SSL fix for downloads, suppress warnings
# Import: torch, torchvision, pytorch3d, tensorboard
# Import: local modules (attack, utils, render, models)
```

What it does: Loads all dependencies and local modules needed for training.

2 `init()` Function (Lines 43-61)

```
def init(detector_attacker, cfg, data_root, args, log):
    # 1. Create data loader for YOLO detection
    # 2. Initialize universal patch
    # 3. Initialize attacker
    # 4. Setup TensorBoard logger
    return data_loader_tsea, vlogger
```

What it does: Sets up the YOLO detection data pipeline and attack components.

Two Data Loaders

The training uses **two separate data loaders**:

Loader	Source	Purpose
--------	--------	---------

Loader	Source	Purpose
person_detection_loader	data/INRIAPerson/Train/pos	Images with people for 2D YOLO detection & attack
background_loader	data/background_trans/background_train_resize	Background images for 3D mesh rendering

```
# Loader 1: Person images for YOLO detection (Line 47, created in init())
person_detection_loader = dataLoader(cfg.DATA.TRAIN.IMG_DIR, ...)

# Loader 2: Background images for 3D rendering (Line 153)
self.background_loader = get_nuscenes_loader(
    img_dir='data/background_trans/background_train_resize', ...
)
```

Training loop iterates over `background_loader` and samples from `person_detection_loader`.

3 PatchTrainer.__init__() (Lines 73-202)

Model Loading (Lines 85-131)

```
if args.arch == "yolov2":      # Load YOLOv2
elif args.arch == "yolov3":    # Load YOLOv3
elif args.arch == "yolov5":    # Load YOLOv5
elif args.arch == "yolov11":   # Load YOLOv11
elif args.arch == "rcnn":      # Load Faster R-CNN
elif args.arch == "deformable-detr": # Load D-DETR
```

Loss Extractor Setup (Lines 139-151)

```
# Each model needs its own probability extractor
self.prob_extractor = YOLOv2MaxProbExtractor(...) # example
self.tv_loss = TotalVariation() # smoothness loss
```

3D Mesh Loading (Lines 177-196)

```
# Load human body parts for rendering
mesh_man = "data/Archive/Man_join/man.obj"
mesh_tshirt = "data/Archive/tshirt_join/tshirt.obj"
mesh_trouser = "data/Archive/trouser_join/trouser.obj"
```

4 PatchTrainer.train() - Main Loop (Lines 222-402)

Setup Phase (Lines 227-240)

```
cfg = ConfigParser(args.cfg)
detector_attacker = UniversalAttacker(cfg, device)
patch = detector_attacker.universal_patch
optimizer = Adam([patch], lr=args.lr)
```

Training Loop (Lines 243-401)

```
for epoch in range(nepoch):
    for i_batch, img_batch in enumerate(train_loader):
```

Step A: 2D Attack (Lines 258-276)

```
# Detect persons in 2D images
all_preds = detector_attacker.detect_bbox(img_tensor_batch)
# Attack to fool detector
patch_loss, patch_tv_loss, patch_det_loss = detector_attacker.attack(...)
```

Step B: 3D Rendering (Lines 278-290)

```
# Apply patch texture to 3D mesh
renderer.set_adv_patch_texture(patch)
# Generate composite images with rendered person on background
composite_images, gts = renderer.generate_composite_image_tensor(bg)
```

Step C: 3D Detection Loss (Lines 322-330)

```
# Run detector on rendered images
output = self.model(p_img_batch)
# Calculate detection loss
det_loss, max_prob_list = self.prob_extractor(output, gts_batch, ...)
```

Step D: Total Loss & Update (Lines 332-358)

```
loss = det_loss + patch_det_loss + patch_tv_loss
loss.backward()
optimizer.step()
patch.clamp(0, 1) # Keep patch valid
```

Step E: Logging (Lines 360-401)

```
# TensorBoard logging every 10 batches
# Save patch every 10 epochs
# Print epoch summary
```

5 Main Entry Point (Lines 404-453)

Key Arguments

Argument	Default	Description
--arch	yolov2	Target detector model
--nepoch	800	Number of training epochs
--lr	0.03	Learning rate
--batch_size	2	Batch size
--cfg	configs/baseline/v2.yaml	Config file
--save_path	results/demo	Output directory

Execution

```
trainer = PatchTrainer(args)
trainer.train()
```

🔄 Training Flow Summary



2. Detect persons (2D) → Get detection loss

3. Apply patch to 3D mesh → Render composite

4. Detect persons (3D) → Get 3D detection loss

5. Compute total loss = 2D + 3D + TV

6. Backprop & update patch

 Loss Components

Loss	Purpose
patch_det_loss	Fool detector on 2D images
det_loss	Fool detector on 3D rendered images
patch_tv_loss	Patch smoothness (Total Variation)

 Quick Start

```
# Train with YOLOv2
python train.py --arch yolov2 --nepoch 800 --save_path results/yolov2

# Train with YOLOv5
python train.py --arch yolov5 --nepoch 800 --save_path results/yolov5
```